

字幕：HcbjFO1mRIw

Blink 的 AI 笔记 · 上傳：2026-07-30 · 長度：12:43

原標題：我开源了一个 Skill，把项目经验沉淀成可复用的知识

來源：YouTube (whisper base 語音轉錄，因為影片無 caption)

STT 評語：可讀，有輕微同音字認字 typo (「高价之」=「高價值」，「採过」=「踩過」，「纯肉」=「純粹」等)。主旨 / 術語 / 結構 99% 保留。

1. 純文字版 (clean transcript)

我做了一个Skill能把AI协作项目里
轻身经历过的高价之经验
自动沉电沉可复用的知识
採过的坑
做过的绝测
跑通的方案误出的洞界
不再只散落在海量的对话里
而是纯肉知识库
被后续的项目直接复用
嗨 你好 我是布丁可
短短一段时间
最近给传统行业的朋友
交付了几个定制Adding的
实在没顾上做视频
这几个月我工作上的高价产数
已经百分之百是用Codex
或者Cloud Code来完成了
AI Agent确实托着我的能力编辑
但同时也带来了新的痛点
看看你有没有也遇到过
例如我最近有个项目要对接非速机器人
刚开始以为很简单一句话就调通了
结果却出现了机器人
是不是收不到消息的问题
我跟AI来回排查了好几轮
才把这个问题解决
后来第2个项目居然又遇到了同样的问题
我突然想起来
这个坑之前不是采购吗
于是我不得不回到之前的项目里
去翻早聊天记录
让AI把之前的解决经验
总结成一篇Mark蛋文档
带到新的项目里才把问题解决
这一次我还能记得起来
自己在哪个项目里遇到过它
要是想不起来呢

如果你同时再用Codex
Cloud Code等多个Agent
可能连该去哪里翻早都不知道
所以我想要的其实很简单
当项目里真正解决了一个问题
就让Agent主动把经验记下来
以后再遇到类似的情况
它能先查找一遍
而不是等我的提醒
如果这条经验不止对当前的项目有用
再把它放进Obsidian No型或者非书
留给其他的项目继续用
这样一来换项目换Agent
不等于经验清零
让你的AI越来越滋生
一开始我用的方法很笨
我让Agent把采坑经验
决策依据目标和计划
全部都进进项目里的同一个Mark蛋文档
每次开工前先读一遍
由竞转就完里面捕
用下来发现真相
但是写着写着问题也来了
所有的内容都挤在一个文件里
越来越像一个大杂惠
每个项目的名字都不一样
有的叫Memory
有的叫Plant
还有的叫Nurn
格式也全靠林厂发挥
几个项目同时推进的时候
别说AI了
自己都有脸对不上靠
不过这个笨办法至少证明了一件事
把项目的经验留下来确实有用
接下来要解决的
只是怎么把它做成一套稳定的规则
于是我做了一个Agent's skill
叫Project the Care
我把它做的事叫做经验知识化
项目里秦深经历过的采坑经验
关键决策探索成果和灵感动件
会先留在项目里真正具备复用价值的
在经过你的确认进入长期知识库
日常记录由Agent的自动完成
Airm这个词指的是旅行者
在路上用石头雷起的路标
前面有人走过这里
留下一对石头
后面的人看到

就知道这条路有人走过
也知道该往哪边走
Project the Caren想做的也是这件事
上一个项目解决过的问题
到了下一个项目
不必再摸黑走一遍
这个也面展示了Project the Caren的完整结构
它分两侧
用8种文档配合9个步则
把一条经验从产生保存
一直送到下一个项目
左边是项目侧
也就是你正在使用的Codex
Codecodeworkbody这些Agent
右边是知识库侧
目前以阶桶Obsetting
Notion和飞书
项目侧记录的是这个项目正在发生什么
知识库侧则保存那些值得长期记录和复用的经验
Project the Caren在中间负责流转
接下来我们沿着这条路走一遍
Project the Caren是一个Agent的Skill
装好以后对Agent的说一句
出实话Project the Caren
Agent会先跟你做一轮互动问答
比如项目的一句话定位
沉淀下来的经验
存到哪个知识库
Obsetting
Notion还是飞书
你也可以暂时不配置知识库
等后面有了具体内容想保存的时候再说
你只需要跟着引导一步步完成
Agent就会自动做好相应的配置
对了不只是新项目
已经进行中的老项目也可以随时进入
出实话后项目规则会写进Agent.MD
以后Agent每次进来都会先按照这条规则做事
同时项目里会多出一个叫Caren的文件家
项目的经验都会存在这里
Caren文件家和项目最重要交付的代码
成果文党
这些真正的工程资产格离开
互不打扰
这些工程资产往往随着项目的交付就交出去了
而项目记得的经验会留在Caren里面
持续为我所用
当项目有了实质的进展
Agent会在Log.MD顶部机一条
做了什么

定了什么
详细内容放在哪里都能
记录在啊
他只写灾药加紫针
单调步超过二十行
保持日质的轻快
为什么要叫紫针
Log为了保持轻量化
只记一条灾药
如果有详细的文党
在附上链接
只向记录详细的那篇文党
这个链接就叫紫针
而紫上的详细文党
就是沉淀下来的知识
就是后面要讲的主题文党
当然就算这次推进没有沉淀主题文党
只要有实质性的动作
Log也会照样记忆比
以后再回来
先少一眼Log
就能知道这段时间发生了什么
不必从忙忙的对话里考古
如果一件事
没法在一个会话里做完
Agent会创建
Rodemapp.md
里面只维护三内信息
里程碑当前焦点
和还没有解决的问题
项目向前走一步
完成的里程碑就被勾掉
当前的焦点在一道下一件事
Rodemapp也不展开长篇细节
某个目标
如果有对应的主题文党
就用紫针纸过去
需要实在顺吞摸刮看响情
Rodemapp是朝前看
告诉你
接下来要做什么
Log是朝后看
告诉你之前发生过什么
两份文党放在一起
换会话
换Agent
甚至换帐号
回来以后也不用把背景
再重新讲一遍
它还可以管住那些很走的AI

有些地方明明已经否决了
它经过几轮又想试一次
有些功能说好放在二期
它一会儿又提出来
Rodemapp把这些决定写清楚以后
人和AI看到的是同一份路线图
项目里经常会收到外部的资料
比如假方给的业务规则
输出模板
或者一份不能改动的原始文党
你把材料给到Agent
它会放进Reference
目楼这里的规则很简单
原件只规党不改写
Agent对材料的理解
以及后来形成的结论
才会进入到主题文党
这样拿句话来自原文
拿句话来自项目的判断
不会混在一起
接下来
看看项目经验是怎么留下来的
比如刚才那个非出机器人的问题
一旦找到原因并验证的结访
Agent就会把结论写进对应的主题文党
如果已经有相关主题了
就在原文党你更新
所以你想知道某个主题
现在的结论看这一篇文党
就够了
格式上主题文党对其了
谷歌的开放知识格式
OKF的规范
即 mark 当家
YAMO from Matter
说白了这些文党
技能让人直接打开看
也方便Agent 检索和更新
不会被锁在某个特定的产品里
这是我一个真实项目里的Karen文件家
已经沉淀了一批主题文党
例如这篇文党
记录的是关于项目知识
自动保存的 no 型的结论
可以看到顶部是
YAMO格式的front matter
也就是笔记的属性
跟着过程和最后确论的方案
下次再遇到相近的问题
Agent 会先在这里来找

而不是等我想起了
某个舅的聊天记录
有些结论不是在聊天里
直接冒出来的
而是藏在设计文党
实施文党贫神意见
或者复盘文党里面
这些过程材料里
一旦形成了稳定的结论
Agent 也会把它收进主题文党
以著名的Superpowers 为例
它会用spex、plans
帮我们细化任务
让工作做得更稳
而Care可以再从这些文党里面
提取可以长期保留的结论
一个负责可靠的完成工作
一个负责成电工作成果
主题文党你的内容首先服务
当前项目
当一条经验已经真实验证过
而且换个项目人家有用
它才适合进入长期知识库
我把这个过程叫做知识毕业
Agent 可以主动提议
你也可以直接对它说
把这条经验毕业到知识库
但有一条规矩不会变
你不脸头
它就不会自动往知识库里面
你确认以后
Agent 会把项目你的局部信息拉掉
整理成一篇可以独立阅读的知识笔记
在更新index
并记录清楚它来自哪个项目
如果结论后来有了变化
可以再次逼
它会更新原来的笔记
不会再负责一份几乎一样的内容
不同的知识库写法不一样
在opc 定理
index 本身就是一篇笔记
知识之间可以用双链进行链接
在逃行里
数据库本身就是index
一条知识对应遗憾
而Notion 本身的属性
就可以用来存放Front Matter
到了飞书
这些内容就会进入到Wake 节点数

总之每个平台都是适配了该平台的特点
虽然内容的形势不同
但是你发起知识毕业的方式是不变的
项目做久了
知识难免会过时
有些地方也会慢慢走一样
所以Career 还有一个体检
你对Agent 说一句做个体检
它就会检查主体文党
有没有相互打架
练结有没有失效
哪些经验一直压在项目里没有处理
最后是知识回流
Agent 动手前
先找一找这件事
有没有现成的结论
如果项目里有对应的主体文党
就直接使用
如果项目里没有
再到知识库里面去查index
真正用上的知识
会在City 的点MD 枝条摘药
和放一个指向知识库的指针
不负质全文
让过去的经验
在新的工作里直接拍上用场
久不者玩
我们回顾一下
日常的舞步由Agent 自动完成
记落个
维护肉的Map
成电主题
从过程文党里去结论
以及开工前
查找就的经验
你主要在出实话
这是毕业和体检时把把关
一句话总结
你平时还是照常的
跟Agent 推项目
经验会自动留下来
说到这里
你可能会问
Project care
和capacity大声的
LM 危体有什么区别
要回答这个问题
我们可以把
计知识分成三种方式来看
传统笔记特点是高体感

高主力
体感就是指这条知识
跟你的连接有多深
主力就是指你把它记下来有多费劲
传统笔记每句话
都是自己想出来
自己写下来的
所以蓝点也在这里
你不但要在老子里
把事情想明白
还得在项目盲完以后
专门留时间去整理
大多数时候
不是记不起
是真的记不动
以LM 危体为代表的
AI自动生成笔记
特点是主力很低
但往往体感也低
你给它一批资料
它会自动拆解成
很多篇相互连接的危体文档
短时间里
知识库就能迅速地变大
我以前的视频你也演识过
看这一堆笔记瞬间爆出来
确实很爽
但我后来冷静下来发现
偷肯烧了不少
笔记也多了
可有些原始资料
我根本没有认真读过
它只是从一个文件家
搬到了更多的markdown里面
这就有点像
顾了一个机器人
替你跑步
他每天跑时工里
你站在旁边看数据
自己的身体
并没有因此变得更好
教程内的资料
还有另外一个问题
人文可能按照一条故事线
从基础一步一步讲到进接
但是当拆解成了很多原子笔记以后
单篇看起来都很完整
你却不知道该从哪篇开始
所以对我来说
LM为体更像一个自动生成的

四有百科
我会用它整理资料
但不会拿它期待自己的学习过程
Project Care 的经验来自你
亲身参与过的项目
日常记录主要由Edgent 完成
所以是高提感
低阻力
Project Care 的来源
不是一批人进去的资料
而是你正在参与的项目
一条知识之所以出现
是因为你真的碰到了问题
你和Edgent 一起做个判断
也看到了最后的结果
所以你再看到这条笔记时
你会知道当时它为什么会有
以及它有什么作用
如果说LM为体整理的是
给到的资料
Project Care 成电的就是做过的事情
现在我已经把Project Care 作为了基础设施
我觉得除了我自己
对大多数的人
应该也会很有用
所以我把它开源了出来
如果你经常和Edgent 搭档做项目
无论个人还是团队
它就是为你准备的
如果你希望做完的项目
变成长期资产
而不是做完就散
它也正是你需要的
不仅是软件开发
像法律
諮詢 策划
这些支持密集型工作
一样会在过程中
展下大量的值得留住的经验
我希望Project Care 能帮你
在做完四的同时
顺手把经验留下来
这些经验不仅可以戴到新的项目里
等你写数值做复盘的时候
也能直接从里面去找到素材
看看自己这段时间到底做了什么
又击累了什么
这个SKU未经反复改了很多轮
里面还有很多小设计
这里就不全部展开了

留给你自己去发现
欢迎你来使用
并提出意见
我们一起来贡献贡献
也欢迎留下点赞
关注我会继续分享
AI加支持管理的更多时间
我们下期接着聊

2. 時間軸版 (timed transcript)

[00:00.000] 我做了一个Skill能把AI协作项目里
[00:02.480] 轻身经历过的高价之经验
[00:04.440] 自动沉电沉可复用的知识
[00:06.280] 探过的坑
[00:07.120] 做过的绝测
[00:07.960] 跑通的方案误出的洞界
[00:09.680] 不再只散落在海量的对话里
[00:11.680] 而是纯肉知识库
[00:12.840] 被后续的项目直接复用
[00:14.560] 嗨 你好 我是布丁可
[00:15.680] 短短一段时间
[00:16.840] 最近给传统行业的朋友
[00:18.320] 交付了几个定制Adding的
[00:19.920] 实在没顾上做视频
[00:21.640] 这几个月我工作上的高价产数
[00:23.840] 已经百分之百是用Codex
[00:25.920] 或者Cloud Code来完成了
[00:27.440] AI Agent确实托着我的能力编辑
[00:30.240] 但同时也带来了新的痛点
[00:32.240] 看看你有没有也遇到过
[00:36.240] 例如我最近有个项目要对接非速机器人
[00:38.880] 刚开始以为很简单一句话就调通了
[00:41.520] 结果却出现了机器人
[00:43.040] 是不是收不到消息的问题
[00:44.680] 我跟AI来回排查了好几轮
[00:46.560] 才把这个问题解决
[00:47.960] 后来第2个项目居然又遇到了同样的问题
[00:50.720] 我突然想起来
[00:51.560] 这个坑之前不是采购吗
[00:53.200] 于是我不得不回到之前的项目里
[00:55.240] 去翻早聊天记录
[00:56.640] 让AI把之前的解决经验
[00:58.520] 总结成一篇Mark蛋文档
[01:00.400] 带到新的项目里才把问题解决
[01:02.440] 这一次我还能记得起来
[01:04.280] 自己在哪个项目里遇到过它
[01:06.200] 要是想不起来呢
[01:07.480] 如果你同时再用Codex
[01:09.160] Cloud Code等多个Agent
[01:11.120] 可能连该去哪里翻早都不知道
[01:13.360] 所以我想要的其实很简单
[01:18.000] 当项目里真正解决了一个问题
[01:20.160] 就让Agent主动把经验记下来
[01:22.320] 以后再遇到类似的情况
[01:24.120] 它能先查找一遍
[01:25.440] 而不是等我的提醒
[01:26.840] 如果这条经验不止对当前的项目有用
[01:29.600] 再把它放进Obsidian No型或者非书
[01:32.440] 留给其他的项目继续用
[01:34.320] 这样一来换项目换Agent
[01:36.440] 不等于经验清零
[01:37.920] 让你的AI越来越滋生
[01:39.880] 一开始我用的方法很笨
[01:41.760] 我让Agent把采坑经验
[01:43.760] 决策依据目标和计划
[01:45.560] 全部都进进项目里的同一个Mark蛋文档
[01:48.280] 每次开工前先读一遍
[01:50.080] 由竟转就完里面捕
[01:51.560] 用下来发现真相
[01:53.160] 但是写着写着问题也来了
[01:55.040] 所有的内容都挤在一个文件里
[01:57.120] 越来越像一个大杂惠

[01:58.640] 每个项目的名字都不一样
[02:00.680] 有的叫Memory
[02:01.640] 有的叫Plant
[02:02.480] 还有的叫Nurn
[02:03.480] 格式也全靠林厂发挥
[02:05.440] 几个项目同时推进的时候
[02:07.200] 别说AI了
[02:08.120] 自己都有脸对不上靠
[02:09.680] 不过这个笨办法至少证明了一件事
[02:12.120] 把项目的经验留下来确实有用
[02:14.440] 接下来要解决的
[02:15.480] 只是怎么把它做成一套稳定的规则
[02:17.920] 于是我做了一个Agent's skill
[02:19.760] 叫Project the Care
[02:21.800] 我把它做的事叫做经验知识化
[02:24.160] 项目里秦深经历过的采坑经验
[02:26.520] 关键决策探索成果和灵感动件
[02:29.160] 会先留在项目里真正具备复用价值的
[02:31.960] 在经过你的确认进入长期知识库
[02:34.640] 日常记录由Agent的自动完成
[02:36.760] Aim这个词指的是旅行者
[02:38.840] 在路上用石头雷起的路标
[02:41.000] 前面有人走过这里
[02:42.240] 留下一对石头
[02:43.240] 后面的人看到
[02:44.360] 就知道这条路有人走过
[02:46.040] 也知道该往哪边走
[02:47.680] Project the Caren想做的也是这件事
[02:50.240] 上一个项目解决过的问题
[02:52.040] 到了下一个项目
[02:52.960] 不必再摸黑走一遍
[02:56.880] 这个也面展示了Project the Caren的完整结构
[03:00.160] 它分两侧
[03:01.040] 用8种文档配合9个步则
[03:03.160] 把一条经验从产生保存
[03:05.720] 一直送到下一个项目
[03:07.160] 左边是项目侧
[03:08.200] 也就是你正在使用的Codex
[03:10.240] Codecodeworkbody这些Agent
[03:12.400] 右边是知识库侧
[03:13.840] 目前以阶桶Obsetting
[03:15.320] Notion和飞书
[03:16.440] 项目侧记录的是这个项目正在发生什么
[03:19.080] 知识库侧则保存那些值得长期记录和复用的经验
[03:22.920] Project the Caren在中间负责流转
[03:25.400] 接下来我们沿着这条路走一遍
[03:29.320] Project the Caren是一个Agent的Skill
[03:31.640] 装好以后对Agent的说一句
[03:34.040] 出实话Project the Caren
[03:37.840] Agent会先跟你做一轮互动问答
[03:40.280] 比如项目的一句话定位
[03:42.000] 沉淀下来的经验
[03:43.120] 存到哪个知识库
[03:44.280] Obsetting
[03:44.920] Notion还是飞书
[03:46.000] 你也可以暂时不配置知识库
[03:47.720] 等后面有了具体内容想保存的时候再说
[03:50.520] 你只需要跟着引导一步步完成
[03:52.520] Agent就会自动做好相应的配置
[03:54.720] 对了不只是新项目
[03:55.920] 已经进行中的老项目也可以随时进入
[03:58.320] 出实话后项目规则会写进Agent.MD
[04:01.520] 以后Agent每次进来都会先按照这条规则做事
[04:05.320] 同时项目里会多出一个叫Caren的文件家

[04:08.440] 项目的经验都会存在这里
[04:10.320] Caren文件家和项目最重要交付的代码
[04:13.320] 成果文党
[04:14.080] 这些真正的工程资产格离开
[04:16.120] 互不打扰
[04:16.920] 这些工程资产往往随着项目的交付就交出去了
[04:20.920] 而项目记得的经验会留在Caren里面
[04:23.720] 持续为我所用
[04:28.120] 当项目有了实质的进展
[04:29.920] Agent会在Log.MD顶部机一条
[04:32.720] 做了什么
[04:33.320] 定了什么
[04:34.120] 详细内容放在哪里都能
[04:35.920] 记录在啊
[04:36.920] 他只写灾药加紫针
[04:38.720] 单调步超过二十行
[04:40.120] 保持日质的轻快
[04:41.520] 为什么要叫紫针
[04:42.920] Log为了保持轻量化
[04:44.320] 只记一条灾药
[04:45.520] 如果有详细的文党
[04:46.720] 在附上链接
[04:47.720] 只向记录详细的那篇文党
[04:49.720] 这个链接就叫紫针
[04:51.320] 而紫上的详细文党
[04:52.720] 就是沉淀下来的知识
[04:54.320] 就是后面要讲的主题文党
[04:56.120] 当然就算这次推进没有沉淀主题文党
[04:58.720] 只要有实质性的动作
[05:00.320] Log也会照样记忆比
[05:01.920] 以后再回来
[05:02.920] 先少一眼Log
[05:04.120] 就能知道这段时间发生了什么
[05:06.120] 不必从忙忙的对话里考古
[05:10.120] 如果一件事
[05:11.120] 没法在一个会话里做完
[05:12.920] Agent会创建
[05:13.920] Rodemapp.md
[05:15.520] 里面只维护三内信息
[05:17.120] 里程碑当前焦点
[05:18.520] 和还没有解决的问题
[05:20.120] 项目向前走一步
[05:21.320] 完成的里程碑就被勾掉
[05:23.120] 当前的焦点在一道下一件事
[05:25.120] Rodemapp也不展开长篇细节
[05:27.520] 某个目标
[05:28.120] 如果有对应的主题文党
[05:29.720] 就用紫针纸过去
[05:30.920] 需要实在顺吞摸刮看响情
[05:33.320] Rodemapp是朝前看
[05:34.720] 告诉你
[05:35.120] 接下来要做什么
[05:36.120] Log是朝后看
[05:37.320] 告诉你之前发生过什么
[05:39.120] 两份文党放在一起
[05:40.520] 换会话
[05:41.320] 换Agent
[05:41.920] 甚至换帐号
[05:43.120] 回来以后也不用把背景
[05:44.720] 再重新讲一遍
[05:45.920] 它还可以管住那些很走的AI
[05:48.120] 有些地方明明已经否决了
[05:49.920] 它经过几轮又想试一次
[05:51.720] 有些功能说好放在二期

[05:53.520] 它一会儿又提出来
[05:54.720] Rodemapp把这些决定写清楚以后
[05:56.920] 人和AI看到的是同一份路线图
[06:01.520] 项目里经常会收到外部的资料
[06:03.920] 比如假方给的业务规则
[06:05.920] 输出模板
[06:06.720] 或者一份不能改动的原始文党
[06:08.720] 你把材料给到Agent
[06:10.120] 它会放进Reference
[06:11.520] 目楼这里的规则很简单
[06:13.120] 原件只规党不改写
[06:14.920] Agent对材料的理解
[06:16.520] 以及后来形成的结论
[06:18.120] 才会进入到主题文党
[06:19.720] 这样拿句话来自原文
[06:21.320] 拿句话来自项目的判断
[06:22.920] 不会混在一起
[06:25.920] 接下来
[06:26.720] 看看项目经验是怎么留下来的
[06:28.720] 比如刚才那个非出机器人的问题
[06:30.920] 一旦找到原因并验证的结访
[06:33.120] Agent就会把结论写进对应的主题文党
[06:35.920] 如果已经有相关主题了
[06:37.520] 就在原文党你更新
[06:39.120] 所以你想知道某个主题
[06:40.520] 现在的结论看这一篇文党
[06:42.320] 就够了
[06:42.720] 格式上主题文党对其了
[06:44.720] 谷歌的开放知识格式
[06:46.320] OKF的规范
[06:47.520] 即 mark 当家
[06:48.520] YAMO from Matter
[06:49.720] 说白了这些文党
[06:50.720] 技能让人直接打开看
[06:52.320] 也方便Agent 检索和更新
[06:54.320] 不会被锁在某个特定的产品里
[06:56.320] 这是我一个真实项目里的Karen文件家
[06:59.120] 已经沉淀了一批主题文党
[07:01.120] 例如这篇文党
[07:01.920] 记录的是关于项目知识
[07:03.720] 自动保存的 no 型的结论
[07:05.520] 可以看到顶部是
[07:06.720] YAMO格式的front matter
[07:08.320] 也就是笔记的属性
[07:10.520] 跟着过程和最后确论的方案
[07:12.720] 下次再遇到相近的问题
[07:14.320] Agent 会先在这里来找
[07:16.120] 而不是等我想起了
[07:17.520] 某个舅的聊天记录
[07:21.120] 有些结论不是在聊天里
[07:22.920] 直接冒出来的
[07:23.720] 而是藏在设计文党
[07:25.120] 实施文党贫神意见
[07:26.720] 或者复盘文党里面
[07:28.120] 这些过程材料里
[07:29.120] 一旦形成了稳定的结论
[07:30.920] Agent 也会把它收进主题文党
[07:33.120] 以著名的Superpowers 为例
[07:35.120] 它会用spex、 plans
[07:36.920] 帮我们细化任务
[07:38.220] 让工作做得更稳
[07:39.520] 而Care可以再从这些文党里面
[07:41.420] 提取可以长期保留的结论
[07:43.520] 一个负责可靠的完成工作

[07:45.320] 一个负责成电工作成果
[07:49.820] 主题文党你的内容首先服务
[07:51.920] 当前项目
[07:52.720] 当一条经验已经真实验证过
[07:55.120] 而且换个项目人家有用
[07:56.620] 它才适合进入长期知识库
[07:58.620] 我把这个过程叫做知识毕业
[08:00.520] Agent 可以主动提议
[08:02.020] 你也可以直接对它说
[08:03.520] 把这条经验毕业到知识库
[08:05.420] 但有一条规矩不会变
[08:06.920] 你不脸头
[08:07.620] 它就不会自动往知识库里面
[08:09.720] 你确认以后
[08:10.620] Agent 会把项目你的局部信息拉掉
[08:13.220] 整理成一篇可以独立阅读的知识笔记
[08:15.920] 在更新index
[08:17.320] 并记录清楚它来自哪个项目
[08:19.320] 如果结论后来有了变化
[08:20.920] 可以再次逼
[08:21.820] 它会更新原来的笔记
[08:23.320] 不会再负一份几乎一样的内容
[08:25.520] 不同的知识库写法不一样
[08:27.220] 在opc 定理
[08:28.220] index 本身就是一篇笔记
[08:30.120] 知识之间可以用双链进行链接
[08:32.720] 在逃行里
[08:33.520] 数据库本身就是index
[08:35.320] 一条知识对应遗憾
[08:36.920] 而Notion 本身的属性
[08:38.220] 就可以用来存放Front Matter
[08:40.020] 到了飞书
[08:40.820] 这些内容就会进入到Wake 节点数
[08:43.020] 总之每个平台都是适配了该平台的特点
[08:46.120] 虽然内容的形势不同
[08:47.720] 但是你发起知识毕业的方式是不变的
[08:53.720] 项目做久了
[08:54.620] 知识难免会过时
[08:56.020] 有些地方也会慢慢走一样
[08:57.720] 所以Career 还有一个体检
[08:59.620] 你对Agent 说一句做个体检
[09:01.520] 它就会检查主体文党
[09:02.920] 有没有相互打架
[09:04.020] 练结有没有失效
[09:05.420] 哪些经验一直压在项目里没有处理
[09:10.620] 最后是知识回流
[09:11.920] Agent 动手前
[09:13.020] 先找一找这件事
[09:14.320] 有没有现成的结论
[09:15.720] 如果项目里有对应的主体文党
[09:17.720] 就直接使用
[09:18.720] 如果项目里没有
[09:19.820] 再到知识库里面去查index
[09:21.820] 真正用上的知识
[09:23.020] 会在City 的点MD 枝条摘药
[09:25.820] 和放一个指向知识库的指针
[09:28.020] 不负质全文
[09:29.020] 让过去的经验
[09:30.020] 在新的工作里直接拍上用场
[09:31.820] 久不者玩
[09:32.620] 我们回顾一下
[09:33.620] 日常的舞步由Agent 自动完成
[09:35.920] 记落个
[09:36.620] 维护肉的Map

[09:37.620] 成电主题
[09:38.620] 从过程文党里去结论
[09:40.320] 以及开工前
[09:41.220] 查找就的经验
[09:42.320] 你主要在出实话
[09:43.720] 这是毕业和体检时把把关
[09:45.920] 一句话总结
[09:46.820] 你平时还是照常的
[09:48.120] 跟Agent 推项目
[09:49.420] 经验会自动留下来
[09:53.420] 说到这里
[09:54.020] 你可能会问
[09:54.920] Project care
[09:55.920] 和capacity大声的
[09:57.120] LM 危体有什么区别
[09:58.920] 要回答这个问题
[09:59.920] 我们可以把
[10:00.620] 计知识分成三种方式来看
[10:03.020] 传统笔记特点是高体感
[10:05.020] 高主力
[10:05.720] 体感就是指这条知识
[10:07.220] 跟你的连接有多深
[10:08.720] 主力就是指你把它记下来有多费劲
[10:11.420] 传统笔记每句话
[10:12.620] 都是自己想出来
[10:13.920] 自己写下来的
[10:14.920] 所以蓝点也在这里
[10:16.220] 你不但要在老子里
[10:17.320] 把事情想明白
[10:18.320] 还得在项目盲完以后
[10:19.920] 专门留时间去整理
[10:21.320] 大多数时候
[10:22.120] 不是记不起
[10:22.920] 是真的记不动
[10:23.920] 以LM 危体为代表的
[10:25.820] AI自动生成笔记
[10:27.220] 特点是主力很低
[10:28.520] 但往往体感也低
[10:29.920] 你给它一批资料
[10:31.020] 它会自动拆解成
[10:32.220] 很多篇相互连接的危体文档
[10:34.420] 短时间内
[10:35.120] 知识库就能迅速地变大
[10:36.720] 我以前的视频你也演识过
[10:38.520] 看这一堆笔记瞬间爆出来
[10:40.320] 确实很爽
[10:41.120] 但我后来冷静下来发现
[10:42.720] 偷肯烧了不少
[10:43.720] 笔记也多了
[10:44.620] 可有些原始资料
[10:45.620] 我根本没有认真读过
[10:47.120] 它只是从一个文件家
[10:48.720] 搬到了更多的markdown里面
[10:50.420] 这就有点像
[10:51.320] 顾了一个机器人
[10:52.320] 替你跑步
[10:53.120] 他每天跑时工里
[10:54.220] 你站在旁边看数据
[10:55.520] 自己的身体
[10:56.320] 并没有因此变得更好
[10:57.720] 教程内的资料
[10:58.620] 还有另外一个问题
[10:59.620] 人文可能按照一条故事线
[11:01.620] 从基础一步一步讲到进接

[11:03.620] 但是当拆解成了很多原子笔记以后
[11:06.320] 单篇看起来都很完整
[11:07.820] 你却不知道该从哪篇开始
[11:09.620] 所以对我来说
[11:10.620] LM为体更像一个自动生成的
[11:12.820] 四有百科
[11:13.620] 我会用它整理资料
[11:14.620] 但不会拿它期待自己的学习过程
[11:16.820] Project Care 的经验来自你
[11:19.020] 轻身参与过的项目
[11:20.320] 日常记录主要由Edgent 完成
[11:22.420] 所以是高提感
[11:23.720] 低阻力
[11:24.320] Project Care 的来源
[11:25.820] 不是一批人进去的资料
[11:27.420] 而是你正在参与的项目
[11:28.920] 一条知识之所以出现
[11:30.420] 是因为你真的碰到了问题
[11:32.120] 你和Edgent 一起做个判断
[11:33.920] 也看到了最后的结果
[11:35.320] 所以你再看到这条笔记时
[11:37.020] 你会知道当时它为什么会有
[11:38.920] 以及它有什么作用
[11:40.120] 如果说LM为体整理的是
[11:42.120] 给到的资料
[11:43.020] Project Care 成电的就是做过的事情
[11:48.320] 现在我已经把Project Care 作为了基础设施
[11:51.620] 我觉得除了我自己
[11:52.820] 对大多数的人
[11:53.620] 应该也会很有用
[11:54.720] 所以我把它开源了出来
[11:56.220] 如果你经常和Edgent 搭档做项目
[11:58.520] 无论个人还是团队
[11:59.820] 它就是为你准备的
[12:00.920] 如果你希望做完的项目
[12:02.320] 变成长期资产
[12:03.420] 而不是做完就散
[12:04.620] 它也正是你需要的
[12:05.820] 不仅是软件开发
[12:07.020] 像法律
[12:07.720] 諮詢 策划
[12:08.720] 这些支持密集型工作
[12:10.220] 一样会在过程中
[12:11.420] 展下大量的值得留住的经验
[12:13.520] 我希望Project Care 能帮你
[12:15.420] 在做完四的同时
[12:16.820] 顺手把经验留下来
[12:18.320] 这些经验不仅可以戴到新的项目里
[12:20.520] 等你写数值做复盘的时候
[12:22.520] 也能直接从里面去找到素材
[12:24.720] 看看自己这段时间到底做了什么
[12:26.720] 又击累了什么
[12:27.720] 这个SKU未经反复改了很多轮
[12:29.920] 里面还有很多小设计
[12:31.220] 这里就不全部展开了
[12:32.620] 留给你自己去发现
[12:34.120] 欢迎你来使用
[12:35.120] 并提出意见
[12:35.920] 我们一起来贡献贡献
[12:37.720] 也欢迎留下点赞
[12:38.920] 关注我会继续分享
[12:40.320] AI加支持管理的更多时间
[12:42.220] 我们下期接着聊

3. 原始 SRT (raw captions)

1
00:00:00,000 --> 00:00:02,480
我做了一个Skill能把AI协作项目里

2
00:00:02,480 --> 00:00:04,440
轻身经历过的高价之经验

3
00:00:04,440 --> 00:00:06,280
自动沉电沉可复用的知识

4
00:00:06,280 --> 00:00:07,120
采过的坑

5
00:00:07,120 --> 00:00:07,960
做过的绝测

6
00:00:07,960 --> 00:00:09,680
跑通的方案误出的洞界

7
00:00:09,680 --> 00:00:11,680
不再只散落在海量的对话里

8
00:00:11,680 --> 00:00:12,840
而是纯肉知识库

9
00:00:12,840 --> 00:00:14,560
被后续的项目直接复用

10
00:00:14,560 --> 00:00:15,680
嗨 你好 我是布丁可

11
00:00:15,680 --> 00:00:16,840
短短一段时间

12
00:00:16,840 --> 00:00:18,320
最近给传统行业的朋友

13
00:00:18,320 --> 00:00:19,920
交付了几个定制Adding的

14
00:00:19,920 --> 00:00:21,640
实在没顾上做视频

15
00:00:21,640 --> 00:00:23,840
这几个月我工作上的高价产数

16
00:00:23,840 --> 00:00:25,920

已经百分之百是用Codex

17

00:00:25,920 --> 00:00:27,440

或者Cloud Code来完成了

18

00:00:27,440 --> 00:00:30,240

AI Agent确实托着我的能力编辑

19

00:00:30,240 --> 00:00:32,240

但同时也带来了新的痛点

20

00:00:32,240 --> 00:00:33,760

看看你有没有也遇到过

21

00:00:36,240 --> 00:00:38,880

例如我最近有个项目要对接非速机器人

22

00:00:38,880 --> 00:00:41,520

刚开始以为很简单一句话就调通了

23

00:00:41,520 --> 00:00:43,040

结果却出现了机器人

24

00:00:43,040 --> 00:00:44,680

是不是收不到消息的问题

25

00:00:44,680 --> 00:00:46,560

我跟AI来回排查了好几轮

26

00:00:46,560 --> 00:00:47,960

才把这个问题解决

27

00:00:47,960 --> 00:00:50,720

后来第2个项目居然又遇到了同样的问题

28

00:00:50,720 --> 00:00:51,560

我突然想起来

29

00:00:51,560 --> 00:00:53,200

这个坑之前不是采购吗

30

00:00:53,200 --> 00:00:55,240

于是我不得不回到之前的项目里

31

00:00:55,240 --> 00:00:56,600

去翻早聊天记录

32

00:00:56,640 --> 00:00:58,520

让AI把之前的解决经验

33

00:00:58,520 --> 00:01:00,400

总结成一篇Mark蛋文档

34

00:01:00,400 --> 00:01:02,440

带到新的项目里才把问题解决

35

00:01:02,440 --> 00:01:04,280

这一次我还能记得起来

36

00:01:04,280 --> 00:01:06,200

自己在哪个项目里遇到过它

37

00:01:06,200 --> 00:01:07,480

要是想不起来呢

38

00:01:07,480 --> 00:01:09,160

如果你同时再用Codex

39

00:01:09,160 --> 00:01:11,120

Cloud Code等多个Agent

40

00:01:11,120 --> 00:01:13,360

可能连该去哪里翻早都不知道

41

00:01:13,360 --> 00:01:15,400

所以我想要的其实很简单

42

00:01:18,000 --> 00:01:20,160

当项目里真正解决了一个问题

43

00:01:20,160 --> 00:01:22,320

就让Agent主动把经验记下来

44

00:01:22,320 --> 00:01:24,080

以后再遇到类似的情况

45

00:01:24,120 --> 00:01:25,440

它能先查找一遍

46

00:01:25,440 --> 00:01:26,840

而不是等我的提醒

47

00:01:26,840 --> 00:01:29,600

如果这条经验不止对当前的项目有用

48

00:01:29,600 --> 00:01:32,440

再把它放进Obsidian No型或者非书

49

00:01:32,440 --> 00:01:34,320

留给其他的项目继续用

50

00:01:34,320 --> 00:01:36,440

这样一来换项目换Agent

51

00:01:36,440 --> 00:01:37,920

不等于经验清零

52

00:01:37,920 --> 00:01:39,880

让你的AI越来越滋生

53

00:01:39,880 --> 00:01:41,760

一开始我用的方法很笨

54

00:01:41,760 --> 00:01:43,760

我让Agent把采坑经验

55

00:01:43,760 --> 00:01:45,560

决策依据目标和计划

56

00:01:45,560 --> 00:01:48,280

全部都进进项目里的同一个Mark蛋文档

57

00:01:48,280 --> 00:01:50,080

每次开工前先读一遍

58

00:01:50,080 --> 00:01:51,560

由竞转就完里面捕

59

00:01:51,560 --> 00:01:53,120

用下来发现真相

60

00:01:53,160 --> 00:01:55,040

但是写着写着问题也来了

61

00:01:55,040 --> 00:01:57,120

所有的内容都挤在一个文件里

62

00:01:57,120 --> 00:01:58,640

越来越像一个大杂惠

63

00:01:58,640 --> 00:02:00,680

每个项目的名字都不一样

64

00:02:00,680 --> 00:02:01,640

有的叫Memory

65

00:02:01,640 --> 00:02:02,480

有的叫Plant

66

00:02:02,480 --> 00:02:03,480

还有的叫Nurn

67

00:02:03,480 --> 00:02:05,440

格式也全靠林厂发挥

68

00:02:05,440 --> 00:02:07,200

几个项目同时推进的时候

69

00:02:07,200 --> 00:02:08,120

别说AI了

70

00:02:08,120 --> 00:02:09,680

自己都有脸对不上靠

71

00:02:09,680 --> 00:02:12,120

不过这个笨办法至少证明了一件事

72

00:02:12,120 --> 00:02:14,440

把项目的经验留下来确实有用

73

00:02:14,440 --> 00:02:15,480

接下来要解决的

74

00:02:15,480 --> 00:02:17,920

只是怎么把它做成一套稳定的规则

75

00:02:17,920 --> 00:02:19,760

于是我做了一个Agent's skill

76

00:02:19,760 --> 00:02:21,400

叫Project the Care

77

00:02:21,800 --> 00:02:24,160

我把它做的事叫做经验知识化

78

00:02:24,160 --> 00:02:26,520

项目里秦深经历过的采坑经验

79

00:02:26,520 --> 00:02:29,160

关键决策探索成果和灵感动件

80

00:02:29,160 --> 00:02:31,960

会先留在项目里真正具备复用价值的

81

00:02:31,960 --> 00:02:34,640

在经过你的确认进入长期知识库

82

00:02:34,640 --> 00:02:36,760

日常记录由Agent的自动完成

83

00:02:36,760 --> 00:02:38,840

Aim这个词指的是旅行者

84

00:02:38,840 --> 00:02:41,000

在路上用石头雷起的路标

85

00:02:41,000 --> 00:02:42,240

前面有人走过这里

86

00:02:42,240 --> 00:02:43,240

留下一对石头

87

00:02:43,240 --> 00:02:44,360

后面的人看到

88

00:02:44,360 --> 00:02:46,040

就知道这条路有人走过

89

00:02:46,040 --> 00:02:47,680

也知道该往哪边走

90

00:02:47,680 --> 00:02:50,240

Project the Caren想做的也是这件事

91

00:02:50,240 --> 00:02:52,040

上一个项目解决过的问题

92

00:02:52,040 --> 00:02:52,960

到了下一个项目

93

00:02:52,960 --> 00:02:54,480

不必再摸黑走一遍

94

00:02:56,880 --> 00:03:00,160

这个也面展示了Project the Caren的完整结构

95

00:03:00,160 --> 00:03:01,040

它分两侧

96

00:03:01,040 --> 00:03:03,160

用8种文档配合9个步则

97

00:03:03,160 --> 00:03:05,720

把一条经验从产生保存

98

00:03:05,720 --> 00:03:07,160

一直送到下一个项目

99

00:03:07,160 --> 00:03:08,200

左边是项目侧

100

00:03:08,200 --> 00:03:10,240

也就是你正在使用的Codex

101

00:03:10,240 --> 00:03:12,400

Codecodeworkbody这些Agent

102

00:03:12,400 --> 00:03:13,840

右边是知识库侧

103

00:03:13,840 --> 00:03:15,320

目前以阶桶Obsetting

104

00:03:15,320 --> 00:03:16,440

Notion和飞书

105

00:03:16,440 --> 00:03:19,080

项目侧记录的是这个项目正在发生什么

106

00:03:19,080 --> 00:03:22,920

知识库侧则保存那些值得长期记录和复用的经验

107

00:03:22,920 --> 00:03:25,400

Project the Caren在中间负责流转

108

00:03:25,400 --> 00:03:27,320

接下来我们沿着这条路走一遍

109

00:03:29,320 --> 00:03:31,640

Project the Caren是一个Agent的Skill

110

00:03:31,640 --> 00:03:34,040

装好以后对Agent的说一句

111

00:03:34,040 --> 00:03:35,840

出实话Project the Caren

112

00:03:37,840 --> 00:03:40,280

Agent会先跟你做一轮互动问答

113

00:03:40,280 --> 00:03:42,000

比如项目的一句话定位

114

00:03:42,000 --> 00:03:43,120

沉淀下来的经验

115

00:03:43,120 --> 00:03:44,280

存到哪个知识库

116

00:03:44,280 --> 00:03:44,920

Obsetting

117

00:03:44,920 --> 00:03:46,000

Notion还是飞书

118

00:03:46,000 --> 00:03:47,720

你也可以暂时不配置知识库

119

00:03:47,720 --> 00:03:50,520

等后面有了具体内容想保存的时候再说

120

00:03:50,520 --> 00:03:52,520

你只需要跟着引导一步步完成

121

00:03:52,520 --> 00:03:54,720

Agent就会自动做好相应的配置

122

00:03:54,720 --> 00:03:55,920

对了不只是新项目

123

00:03:55,920 --> 00:03:58,320

已经进行中的老项目也可以随时进入

124

00:03:58,320 --> 00:04:01,520

出实话后项目规则会写进Agent.MD

125

00:04:01,520 --> 00:04:05,320

以后Agent每次进来都会先按照这条规则做事

126

00:04:05,320 --> 00:04:08,440

同时项目里会多出一个叫Caren的文件家

127

00:04:08,440 --> 00:04:10,320

项目的经验都会存在这里

128

00:04:10,320 --> 00:04:13,320

Caren文件家和项目最重要交付的代码

129

00:04:13,320 --> 00:04:14,080

成果文党

130

00:04:14,080 --> 00:04:16,120

这些真正的工程资产格离开

131

00:04:16,120 --> 00:04:16,920

互不打扰

132

00:04:16,920 --> 00:04:20,920

这些工程资产往往随着项目的交付就交出去了

133

00:04:20,920 --> 00:04:23,720

而项目记得的经验会留在Caren里面

134

00:04:23,720 --> 00:04:25,120

持续为我所用

135

00:04:28,120 --> 00:04:29,920

当项目有了实质的进展

136

00:04:29,920 --> 00:04:32,720

Agent会在Log.MD顶部机一条

137

00:04:32,720 --> 00:04:33,320

做了什么

138

00:04:33,320 --> 00:04:34,120

定了什么

139

00:04:34,120 --> 00:04:35,920

详细内容放在哪里都能

140

00:04:35,920 --> 00:04:36,920

记录在啊

141

00:04:36,920 --> 00:04:38,720

他只写灾药加紫针

142

00:04:38,720 --> 00:04:40,120

单调步超过二十行

143

00:04:40,120 --> 00:04:41,520

保持日质的轻快

144

00:04:41,520 --> 00:04:42,920

为什么要叫紫针

145

00:04:42,920 --> 00:04:44,320

Log为了保持轻量化

146

00:04:44,320 --> 00:04:45,520

只记一条灾药

147

00:04:45,520 --> 00:04:46,720

如果有详细的文党

148

00:04:46,720 --> 00:04:47,720

在附上链接

149

00:04:47,720 --> 00:04:49,720

只向记录详细的那篇文党

150

00:04:49,720 --> 00:04:51,320

这个链接就叫紫针

151

00:04:51,320 --> 00:04:52,720

而紫上的详细文党

152

00:04:52,720 --> 00:04:54,320

就是沉淀下来的知识

153

00:04:54,320 --> 00:04:56,120

就是后面要讲的主题文党

154

00:04:56,120 --> 00:04:58,720

当然就算这次推进没有沉淀主题文党

155

00:04:58,720 --> 00:05:00,320

只要有实质性的动作

156

00:05:00,320 --> 00:05:01,920

Log也会照样记忆比

157

00:05:01,920 --> 00:05:02,920

以后再回来

158

00:05:02,920 --> 00:05:04,120

先少一眼Log

159

00:05:04,120 --> 00:05:06,120

就能知道这段时间发生了什么

160

00:05:06,120 --> 00:05:08,120

不必从忙忙的对话里考古

161

00:05:10,120 --> 00:05:11,120

如果一件事

162

00:05:11,120 --> 00:05:12,920

没法在一个会话里做完

163

00:05:12,920 --> 00:05:13,920

Agent会创建

164

00:05:13,920 --> 00:05:15,520

Rodemapp.md

165

00:05:15,520 --> 00:05:17,120

里面只维护三内信息

166

00:05:17,120 --> 00:05:18,520

里程碑当前焦点

167

00:05:18,520 --> 00:05:20,120

和还没有解决的问题

168

00:05:20,120 --> 00:05:21,320

项目向前走一步

169

00:05:21,320 --> 00:05:23,120

完成的里程碑就被勾掉

170

00:05:23,120 --> 00:05:25,120

当前的焦点在一道下一件事

171

00:05:25,120 --> 00:05:27,520

Rodemapp也不展开长篇细节

172

00:05:27,520 --> 00:05:28,120

某个目标

173

00:05:28,120 --> 00:05:29,720

如果有对应的主题文党

174

00:05:29,720 --> 00:05:30,920

就用紫针纸过去

175

00:05:30,920 --> 00:05:33,320

需要实在顺吞摸刮看响情

176

00:05:33,320 --> 00:05:34,720

Rodemapp是朝前看

177

00:05:34,720 --> 00:05:35,120

告诉你

178

00:05:35,120 --> 00:05:36,120

接下来要做什么

179

00:05:36,120 --> 00:05:37,320

Log是朝后看

180

00:05:37,320 --> 00:05:39,120

告诉你之前发生过什么

181

00:05:39,120 --> 00:05:40,520

两份文党放在一起

182

00:05:40,520 --> 00:05:41,320

换会话

183

00:05:41,320 --> 00:05:41,920

换Agent

184

00:05:41,920 --> 00:05:43,120

甚至换帐号

185

00:05:43,120 --> 00:05:44,720

回来以后也不用把背景

186

00:05:44,720 --> 00:05:45,920

再重新讲一遍

187

00:05:45,920 --> 00:05:48,120

它还可以管住那些很走的AI

188

00:05:48,120 --> 00:05:49,920

有些地方明明已经否决了

189

00:05:49,920 --> 00:05:51,720

它经过几轮又想试一次

190

00:05:51,720 --> 00:05:53,520

有些功能说好放在二期

191

00:05:53,520 --> 00:05:54,720

它一会儿又提出来

192

00:05:54,720 --> 00:05:56,920

Rodemapp把这些决定写清楚以后

193

00:05:56,920 --> 00:05:59,720

人和AI看到的是同一份路线图

194

00:06:01,520 --> 00:06:03,920

项目里经常会收到外部的资料

195

00:06:03,920 --> 00:06:05,920

比如假方给的业务规则

196

00:06:05,920 --> 00:06:06,720

输出模板

197

00:06:06,720 --> 00:06:08,720

或者一份不能改动的原始文档

198

00:06:08,720 --> 00:06:10,120

你把材料给到Agent

199

00:06:10,120 --> 00:06:11,520

它会放进Reference

200

00:06:11,520 --> 00:06:13,120

目录这里的规则很简单

201

00:06:13,120 --> 00:06:14,920

原件只规定不改写

202

00:06:14,920 --> 00:06:16,520

Agent对材料的理解

203

00:06:16,520 --> 00:06:18,120

以及后来形成的结论

204

00:06:18,120 --> 00:06:19,720

才会进入到主题文档

205

00:06:19,720 --> 00:06:21,320

这样拿句话来自原文

206

00:06:21,320 --> 00:06:22,920

拿句话来自项目的判断

207

00:06:22,920 --> 00:06:23,920

不会混在一起

208

00:06:25,920 --> 00:06:26,720

接下来

209

00:06:26,720 --> 00:06:28,720

看看项目经验是怎么留下来的

210

00:06:28,720 --> 00:06:30,920

比如刚才那个非出机器人的问题

211

00:06:30,920 --> 00:06:33,120

一旦找到原因并验证的结访

212

00:06:33,120 --> 00:06:35,920

Agent就会把结论写进对应的主题文党

213

00:06:35,920 --> 00:06:37,520

如果已经有相关主题了

214

00:06:37,520 --> 00:06:39,120

就在原文党你更新

215

00:06:39,120 --> 00:06:40,520

所以你想知道某个主题

216

00:06:40,520 --> 00:06:42,320

现在的结论看这一篇文党

217

00:06:42,320 --> 00:06:42,720

就够了

218

00:06:42,720 --> 00:06:44,720

格式上主题文党对其了

219

00:06:44,720 --> 00:06:46,320

谷歌的开放知识格式

220

00:06:46,320 --> 00:06:47,520

OKF的规范

221

00:06:47,520 --> 00:06:48,520

即 mark 当家

222

00:06:48,520 --> 00:06:49,720

YAMO from Matter

223

00:06:49,720 --> 00:06:50,720

说白了这些文党

224

00:06:50,720 --> 00:06:52,320

技能让人直接打开看

225

00:06:52,320 --> 00:06:54,320

也方便Agent 检索和更新

226

00:06:54,320 --> 00:06:56,320

不会被锁在某个特定的产品里

227

00:06:56,320 --> 00:06:59,120

这是我一个真实项目里的Karen文件家

228

00:06:59,120 --> 00:07:01,120

已经沉淀了一批主题文党

229

00:07:01,120 --> 00:07:01,920

例如这篇文党

230

00:07:01,920 --> 00:07:03,720

记录的是关于项目知识

231

00:07:03,720 --> 00:07:05,520

自动保存的 no 型的结论

232

00:07:05,520 --> 00:07:06,720

可以看到顶部是

233

00:07:06,720 --> 00:07:08,320

YAMO格式的front matter

234

00:07:08,320 --> 00:07:09,520

也就是笔记的属性

235

00:07:10,520 --> 00:07:12,720

跟着过程和最后确论的方案

236

00:07:12,720 --> 00:07:14,320

下次再遇到相近的问题

237

00:07:14,320 --> 00:07:16,120

Agent 会先在这里来找

238

00:07:16,120 --> 00:07:17,520

而不是等我想起了

239

00:07:17,520 --> 00:07:18,920

某个舅的聊天记录

240

00:07:21,120 --> 00:07:22,920

有些结论不是在聊天里

241

00:07:22,920 --> 00:07:23,720
直接冒出来的

242

00:07:23,720 --> 00:07:25,120
而是藏在设计文党

243

00:07:25,120 --> 00:07:26,720
实施文党贫神意见

244

00:07:26,720 --> 00:07:28,120
或者复盘文党里面

245

00:07:28,120 --> 00:07:29,120
这些过程材料里

246

00:07:29,120 --> 00:07:30,920
一旦形成了稳定的结论

247

00:07:30,920 --> 00:07:33,120
Agent 也会把它收进主题文党

248

00:07:33,120 --> 00:07:35,120
以著名的Superpowers 为例

249

00:07:35,120 --> 00:07:36,920
它会用spex、 plans

250

00:07:36,920 --> 00:07:38,220
帮我们细化任务

251

00:07:38,220 --> 00:07:39,420
让工作做得更稳

252

00:07:39,520 --> 00:07:41,420
而Care可以再从这些文党里面

253

00:07:41,420 --> 00:07:43,520
提取可以长期保留的结论

254

00:07:43,520 --> 00:07:45,320
一个负责可靠的完成工作

255

00:07:45,320 --> 00:07:47,020
一个负责成电工作成果

256

00:07:49,820 --> 00:07:51,920

主题文党的内容首先服务

257

00:07:51,920 --> 00:07:52,720

当前项目

258

00:07:52,720 --> 00:07:55,120

当一条经验已经真实验证过

259

00:07:55,120 --> 00:07:56,620

而且换个项目人家有用

260

00:07:56,620 --> 00:07:58,620

它才适合进入长期知识库

261

00:07:58,620 --> 00:08:00,520

我把这个过程叫做知识毕业

262

00:08:00,520 --> 00:08:02,020

Agent 可以主动提议

263

00:08:02,020 --> 00:08:03,520

你也可以直接对它说

264

00:08:03,520 --> 00:08:05,420

把这条经验毕业到知识库

265

00:08:05,420 --> 00:08:06,920

但有一条规矩不会变

266

00:08:06,920 --> 00:08:07,620

你不脸头

267

00:08:07,620 --> 00:08:09,320

它就不会自动往知识库里面

268

00:08:09,720 --> 00:08:10,620

你确认以后

269

00:08:10,620 --> 00:08:13,220

Agent 会把项目你的局部信息拉掉

270

00:08:13,220 --> 00:08:15,920

整理成一篇可以独立阅读的知识笔记

271

00:08:15,920 --> 00:08:17,320

在更新index

272

00:08:17,320 --> 00:08:19,320

并记录清楚它来自哪个项目

273

00:08:19,320 --> 00:08:20,920

如果结论后来有了变化

274

00:08:20,920 --> 00:08:21,820

可以再次逼

275

00:08:21,820 --> 00:08:23,320

它会更新原来的笔记

276

00:08:23,320 --> 00:08:25,520

不会再负质一份几乎一样的内容

277

00:08:25,520 --> 00:08:27,220

不同的知识库写法不一样

278

00:08:27,220 --> 00:08:28,220

在opc 定理

279

00:08:28,220 --> 00:08:30,120

index 本身就是一篇笔记

280

00:08:30,120 --> 00:08:32,720

知识之间可以用双链进行链接

281

00:08:32,720 --> 00:08:33,520

在逃行里

282

00:08:33,520 --> 00:08:35,320

数据库本身就是index

283

00:08:35,320 --> 00:08:36,820

一条知识对应遗憾

284

00:08:36,920 --> 00:08:38,220

而Notion 本身的属性

285

00:08:38,220 --> 00:08:40,020

就可以用来存放Front Matter

286

00:08:40,020 --> 00:08:40,820

到了飞书

287

00:08:40,820 --> 00:08:43,020

这些内容就会进入到Wake 节点数

288

00:08:43,020 --> 00:08:46,120

总之每个平台都是适配了该平台的特点

289

00:08:46,120 --> 00:08:47,720

虽然内容的形势不同

290

00:08:47,720 --> 00:08:50,720

但是你发起知识毕业的方式是不变的

291

00:08:53,720 --> 00:08:54,620

项目做久了

292

00:08:54,620 --> 00:08:56,020

知识难免会过时

293

00:08:56,020 --> 00:08:57,720

有些地方也会慢慢走一样

294

00:08:57,720 --> 00:08:59,620

所以Career 还有一个体检

295

00:08:59,620 --> 00:09:01,520

你对Agent 说一句做个体检

296

00:09:01,520 --> 00:09:02,920

它就会检查主体文党

297

00:09:02,920 --> 00:09:04,020

有没有相互打架

298

00:09:04,020 --> 00:09:05,320

练结有没有失效

299

00:09:05,420 --> 00:09:07,820

哪些经验一直压在项目里没有处理

300

00:09:10,620 --> 00:09:11,920

最后是知识回流

301

00:09:11,920 --> 00:09:13,020

Agent 动手前

302

00:09:13,020 --> 00:09:14,320

先找一找这件事

303

00:09:14,320 --> 00:09:15,720

有没有现成的结论

304

00:09:15,720 --> 00:09:17,720

如果项目里有对应的主体文党

305

00:09:17,720 --> 00:09:18,720

就直接使用

306

00:09:18,720 --> 00:09:19,820

如果项目里没有

307

00:09:19,820 --> 00:09:21,820

再到知识库里面去查index

308

00:09:21,820 --> 00:09:23,020

真正用上的知识

309

00:09:23,020 --> 00:09:25,820

会在City 的点MD 技条摘药

310

00:09:25,820 --> 00:09:28,020

和放一个指向知识库的指针

311

00:09:28,020 --> 00:09:29,020

不负质全文

312

00:09:29,020 --> 00:09:30,020

让过去的经验

313

00:09:30,020 --> 00:09:31,820

在新的工作里直接拍上用场

314

00:09:31,820 --> 00:09:32,620

久不者玩

315

00:09:32,620 --> 00:09:33,620

我们回顾一下

316

00:09:33,620 --> 00:09:35,920

日常的舞步由Agent 自动完成

317

00:09:35,920 --> 00:09:36,620

记落个

318

00:09:36,620 --> 00:09:37,620

维护肉的Map

319

00:09:37,620 --> 00:09:38,620

成电主题

320

00:09:38,620 --> 00:09:40,320

从过程文党里去结论

321

00:09:40,320 --> 00:09:41,220
以及开工前

322

00:09:41,220 --> 00:09:42,320
查找就的经验

323

00:09:42,320 --> 00:09:43,720
你主要在出实话

324

00:09:43,720 --> 00:09:45,920
这是毕业和体检时把把关

325

00:09:45,920 --> 00:09:46,820
一句话总结

326

00:09:46,820 --> 00:09:48,120
你平时还是照常的

327

00:09:48,120 --> 00:09:49,420
跟Agent 推项目

328

00:09:49,420 --> 00:09:50,820
经验会自动留下来

329

00:09:53,420 --> 00:09:54,020
说到这里

330

00:09:54,020 --> 00:09:54,920
你可能会问

331

00:09:54,920 --> 00:09:55,920
Project care

332

00:09:55,920 --> 00:09:57,100
和capacity大声的

333

00:09:57,120 --> 00:09:58,920
LM 危体有什么区别

334

00:09:58,920 --> 00:09:59,920
要回答这个问题

335

00:09:59,920 --> 00:10:00,620
我们可以把

336

00:10:00,620 --> 00:10:02,920

计知识分成三种方式来看

337

00:10:03,020 --> 00:10:05,020

传统笔记特点是高体感

338

00:10:05,020 --> 00:10:05,720

高主力

339

00:10:05,720 --> 00:10:07,220

体感就是指这条知识

340

00:10:07,220 --> 00:10:08,720

跟你的连接有多深

341

00:10:08,720 --> 00:10:11,420

主力就是指你把它记下来有多费劲

342

00:10:11,420 --> 00:10:12,620

传统笔记每句话

343

00:10:12,620 --> 00:10:13,920

都是自己想出来

344

00:10:13,920 --> 00:10:14,920

自己写下来的

345

00:10:14,920 --> 00:10:16,220

所以蓝点也在这里

346

00:10:16,220 --> 00:10:17,320

你不但要在老子里

347

00:10:17,320 --> 00:10:18,320

把事情想明白

348

00:10:18,320 --> 00:10:19,920

还得在项目盲完以后

349

00:10:19,920 --> 00:10:21,320

专门留时间去整理

350

00:10:21,320 --> 00:10:22,120

大多数时候

351

00:10:22,120 --> 00:10:22,920

不是记不起

352

00:10:22,920 --> 00:10:23,920

是真的记不动

353

00:10:23,920 --> 00:10:25,820

以LM 危体为代表的

354

00:10:25,820 --> 00:10:27,220

AI自动生成笔记

355

00:10:27,220 --> 00:10:28,520

特点是主力很低

356

00:10:28,520 --> 00:10:29,920

但往往体感也低

357

00:10:29,920 --> 00:10:31,020

你给它一批资料

358

00:10:31,020 --> 00:10:32,220

它会自动拆解成

359

00:10:32,220 --> 00:10:34,420

很多篇相互连接的危体文档

360

00:10:34,420 --> 00:10:35,120

短时间里

361

00:10:35,120 --> 00:10:36,720

知识库就能迅速地变大

362

00:10:36,720 --> 00:10:38,520

我以前的视频你也演识过

363

00:10:38,520 --> 00:10:40,320

看这一堆笔记瞬间爆出来

364

00:10:40,320 --> 00:10:41,120

确实很爽

365

00:10:41,120 --> 00:10:42,720

但我后来冷静下来发现

366

00:10:42,720 --> 00:10:43,720

偷肯烧了不少

367

00:10:43,720 --> 00:10:44,620

笔记也多了

368

00:10:44,620 --> 00:10:45,620

可有些原始资料

369

00:10:45,620 --> 00:10:47,120

我根本没有认真读过

370

00:10:47,120 --> 00:10:48,720

它只是从一个文件家

371

00:10:48,720 --> 00:10:50,420

搬到了更多的markdown里面

372

00:10:50,420 --> 00:10:51,320

这就有点像

373

00:10:51,320 --> 00:10:52,320

顾了一个机器人

374

00:10:52,320 --> 00:10:53,120

替你跑步

375

00:10:53,120 --> 00:10:54,220

他每天跑时工里

376

00:10:54,220 --> 00:10:55,520

你站在旁边看数据

377

00:10:55,520 --> 00:10:56,320

自己的身体

378

00:10:56,320 --> 00:10:57,720

并没有因此变得更好

379

00:10:57,720 --> 00:10:58,620

教程内的资料

380

00:10:58,620 --> 00:10:59,620

还有另外一个问题

381

00:10:59,620 --> 00:11:01,620

人文可能按照一条故事线

382

00:11:01,620 --> 00:11:03,620

从基础一步一步讲到进接

383

00:11:03,620 --> 00:11:06,320

但是当拆解成了很多原子笔记以后

384

00:11:06,320 --> 00:11:07,820

单篇看起来都很完整

385

00:11:07,820 --> 00:11:09,620

你却不知道该从哪篇开始

386

00:11:09,620 --> 00:11:10,620

所以对我来说

387

00:11:10,620 --> 00:11:12,820

LM为体更像一个自动生成的

388

00:11:12,820 --> 00:11:13,620

四有百科

389

00:11:13,620 --> 00:11:14,620

我会用它整理资料

390

00:11:14,620 --> 00:11:16,820

但不会拿它期待自己的学习过程

391

00:11:16,820 --> 00:11:19,020

Project Care 的经验来自你

392

00:11:19,020 --> 00:11:20,320

轻身参与过的项目

393

00:11:20,320 --> 00:11:22,420

日常记录主要由Edgent 完成

394

00:11:22,420 --> 00:11:23,720

所以是高提感

395

00:11:23,720 --> 00:11:24,320

低阻力

396

00:11:24,320 --> 00:11:25,820

Project Care 的来源

397

00:11:25,820 --> 00:11:27,320

不是一批人进去的资料

398

00:11:27,420 --> 00:11:28,920

而是你正在参与的项目

399

00:11:28,920 --> 00:11:30,420

一条知识之所以出现

400

00:11:30,420 --> 00:11:32,120

是因为你真的碰到了问题

401

00:11:32,120 --> 00:11:33,920

你和Edgent 一起做个判断

402

00:11:33,920 --> 00:11:35,320

也看到了最后的结果

403

00:11:35,320 --> 00:11:37,020

所以你再看到这条笔记时

404

00:11:37,020 --> 00:11:38,920

你会知道当时它为什么会有

405

00:11:38,920 --> 00:11:40,120

以及它有什么作用

406

00:11:40,120 --> 00:11:42,120

如果说LM为体整理的是

407

00:11:42,120 --> 00:11:43,020

给到的资料

408

00:11:43,020 --> 00:11:45,720

Project Care 成电的就是做过的事情

409

00:11:48,320 --> 00:11:51,620

现在我已经把Project Care 作为了基础设施

410

00:11:51,620 --> 00:11:52,820

我觉得除了我自己

411

00:11:52,820 --> 00:11:53,620

对大多数的人

412

00:11:53,620 --> 00:11:54,720

应该也会很有用

413

00:11:54,720 --> 00:11:56,120

所以我把它开源了出来

414

00:11:56,220 --> 00:11:58,520

如果你经常和Edgent 搭档做项目

415

00:11:58,520 --> 00:11:59,820

无论个人还是团队

416

00:11:59,820 --> 00:12:00,920

它就是为你准备的

417

00:12:00,920 --> 00:12:02,320

如果你希望做完的项目

418

00:12:02,320 --> 00:12:03,420

变成长期资产

419

00:12:03,420 --> 00:12:04,620

而不是做完就散

420

00:12:04,620 --> 00:12:05,820

它也正是你需要的

421

00:12:05,820 --> 00:12:07,020

不仅是软件开发

422

00:12:07,020 --> 00:12:07,720

像法律

423

00:12:07,720 --> 00:12:08,720

諮詢 策划

424

00:12:08,720 --> 00:12:10,220

这些支持密集型工作

425

00:12:10,220 --> 00:12:11,420

一样会在过程中

426

00:12:11,420 --> 00:12:13,520

展下大量的值得留住的经验

427

00:12:13,520 --> 00:12:15,420

我希望Project Care 能帮你

428

00:12:15,420 --> 00:12:16,820

在做完四的同时

429

00:12:16,820 --> 00:12:18,320

顺手把经验留下来

430

00:12:18,320 --> 00:12:20,520

这些经验不仅可以戴到新的项目里

431

00:12:20,520 --> 00:12:22,520

等你写数值做复盘的时候

432

00:12:22,520 --> 00:12:24,620

也能直接从里面去找到素材

433

00:12:24,720 --> 00:12:26,720

看看自己这段时间到底做了什么

434

00:12:26,720 --> 00:12:27,720

又击累了什么

435

00:12:27,720 --> 00:12:29,920

这个SKU未经反复改了很多轮

436

00:12:29,920 --> 00:12:31,220

里面还有很多小设计

437

00:12:31,220 --> 00:12:32,620

这里就不全部展开了

438

00:12:32,620 --> 00:12:34,120

留给你自己去发现

439

00:12:34,120 --> 00:12:35,120

欢迎你来使用

440

00:12:35,120 --> 00:12:35,920

并提出意见

441

00:12:35,920 --> 00:12:37,720

我们一起来贡献贡献

442

00:12:37,720 --> 00:12:38,920

也欢迎留下点赞

443

00:12:38,920 --> 00:12:40,320

关注我会继续分享

444

00:12:40,320 --> 00:12:42,220

AI加支持管理的更多时间

445

00:12:42,220 --> 00:12:43,120

我们下期接着聊